# Algorithms: Bubble Sort Implementation

Lesson Handout & Study Guide — Step-by-Step Sorting & Logic

## 1. Introduction to Sorting & Real-World Examples

Sorting is the process of arranging a collection of items into a specific order (such as ascending or descending). We use sorting every day without even thinking about it!

- **Supermarket Shelves:** Products arranged by weight or price.
- **Train Seats:** Seat numbers arranged sequentially on tickets.
- **School Backpack:** Textbooks sorted by weight—putting the heaviest books at the bottom to balance your bag!

> 💡 **Why Is It Called "Bubble Sort"?**
>
> In Bubble Sort, larger numbers gradually **'bubble up'** to the end of the list with each pass—just like bubbles rising to the surface in a cold glass of *nimbu pani*!

## 2. Understanding Swapping: The "Temp Glass" Trick

Suppose you have two full glasses: one containing **Maaza** and another containing **Milk**. How can you swap their contents without mixing them together or spilling?

You need a third, empty glass called a **Temp (Temporary) glass**!

1. Pour **Maaza** into the **Temp** glass.
2. Pour **Milk** into the now-empty **Maaza** glass.
3. Pour **Maaza** (from the **Temp** glass) into the **Milk** glass.

> ⚠️ **What Happens Without a Temp Variable?**
>
> If you try to overwrite `A = B` directly without saving A first, the original value of A gets overwritten and is **LOST forever**!
>
> *Example:* If `A = 5` and `B = 3`, setting `A = B` results in `A = 3` and `B = 3`. The number 5 disappears completely!

## 3. Tracing Bubble Sort (Pass 1 Step-by-Step)

Let's trace the algorithm on a mixed list of numbers: `[42, 18, 7, 99, 45]`.

| Step / Comparison | Action Taken | Resulting List State |
|---|---|---|
| Compare **42** & **18** | 42 > 18 → **Swap!** | `[18, 42, 7, 99, 45]` |
| Compare **42** & **7** | 42 > 7 → **Swap!** | `[18, 7, 42, 99, 45]` |
| Compare **42** & **99** | 42 < 99 → **No Swap** | `[18, 7, 42, 99, 45]` |

| Step / Comparison | Action Taken | Resulting List State |
|---|---|---|
| Compare **99** & **45** | 99 > 45 → **Swap!** | `[18, 7, 42, 45, 99]` |

*Notice how the largest number (99) successfully "bubbled up" to its correct position at the very end of the list after Pass 1!*

## 4. Algorithmic Logic & Pseudo-Code

To implement Bubble Sort programmatically, we use nested loops:

```
function bubbleSort(scores):
    n = length(scores)
    // Outer loop runs (n - 1) times
    for i from 0 to n - 2:
        // Inner loop compares adjacent elements
        for j from 0 to n - i - 2:
            if scores[j] > scores[j + 1]:
                // The Temp Swap Trick
                temp = scores[j]
                scores[j] = scores[j + 1]
                scores[j + 1] = temp
    return scores
```

**Key Takeaways:**

- Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- A temporary variable (`temp`) is essential during a swap to avoid losing data.
- After pass `i`, the `i`-th largest element is guaranteed to be in its correct final position.

## Practice Questions & Assessment

Test your understanding of the concepts covered in this lesson!

### Question 1: Conceptual Understanding

Why is the algorithm named "Bubble Sort"? Describe the physical analogy discussed in class.

### Question 2: The Temp Variable

What happens if a developer attempts to swap two variable values (e.g., `A = 5` and `B = 3`) using only `A = B; B = A;` without a temporary variable?

A) The values swap correctly without any issue.

B) The original value of A is lost forever, resulting in both variables holding 3.

C) The system throws a compiler syntax error.

D) The list automatically creates an additional element.

### Question 3: Manual Execution / Tracing

Given the initial list of numbers: `[35, 12, 89, 24, 6]`

Perform **Pass 1** of Bubble Sort step-by-step. Show the state of the list after every comparison and swap, and indicate which number reaches its final sorted position at the end of Pass 1.

### Question 4: Real-World Applications

List three everyday real-world examples where sorting helps organize objects or information efficiently.

### Question 5: Code Analysis & Debugging

Consider the following incorrect swap block written by a student:

```
temp = item[j + 1]
item[j + 1] = item[j]
item[j] = item[j + 1]
```

Explain the bug in this code snippet and write the corrected 3-line swap sequence using `temp`.